STUDENT ATTENDANCE SYSTEM

A Python-Based Real-Life Problem Solving Project

PROJECT REPORT

Technology: Python 3 · SQLite · Command-Line Interface · CSV · unittest

Project domain: Education and academic administration

Project type: Student registration and attendance management application

Execution: Local/offline application; standard library only

Student Name: _____

Roll Number: _____

Class / Department: _____

Institution: _____

Academic Year: _____

1. Abstract

The Student Attendance System is a Python application that digitizes basic attendance record management for a school or college. It allows an operator to register students, record daily attendance as Present, Absent, or Late, view date-wise and class-wise reports, calculate individual attendance percentages, and export records to CSV. SQLite provides persistent local storage.

2. Problem Statement

Manual attendance registers require repetitive writing and manual counting. Searching historical records and calculating attendance percentages can take time and may introduce errors. A simple digital system can make attendance recording more organized, searchable, and consistent.

3. Objectives

• Register students using unique student IDs.
• Record each student's daily attendance status.
• Prevent duplicate attendance records for the same student and date.
• Generate date-wise and class-wise attendance reports.
• Calculate an individual student's attendance percentage.
• Export attendance data in a spreadsheet-compatible CSV format.

4. Scope and Intended Users

The project is intended as a small academic demonstration for faculty or administrative staff managing a class. It is a local, single-operator application. It does not include institutional authentication, cloud synchronization, biometric verification, or integration with an official student information system.

5. Tools and Technologies

Python 3.9+ is used for application logic and the command-line interface. SQLite is accessed through Python's built-in sqlite3 module. argparse parses commands, csv exports reports, datetime validates dates, and unittest provides automated testing. No third-party packages are required.

6. System Design

The operator enters a command in the terminal. argparse parses the command and arguments. Application functions validate the inputs and execute database operations. The SQLite database stores student and attendance records. Reports use a join between the two tables; summary calculations count each attendance status and calculate the present percentage.

7. Database Design

The students table stores student_id (primary key), name, class_name, and created_at. The attendance table stores an auto-increment id, student_id (foreign key), attendance_date, status, remarks, and created_at. A database CHECK constraint restricts status values to Present, Absent, or Late. A UNIQUE constraint on student_id and attendance_date prevents duplicate daily attendance.

8. Main Modules

Student registration validates required fields and unique IDs. Attendance marking verifies that the student exists, validates the date and status, and inserts one daily record. Reporting supports optional date and class filters. Student summary calculates total sessions, present, absent, late, and attendance percentage. CSV export writes the selected report records to a file.

9. Execution Instructions

Run commands from the project root using Python. The database is automatically created at data/attendance.db on first use. Example commands are provided below.

Example commands

```
python -m attendance_system.app add-student --id S001 --name "Aarav Shah" --class FY-BSc
python -m attendance_system.app mark --id S001 --status Present --date 2026-11-05
python -m attendance_system.app report --date 2026-11-05
python -m attendance_system.app summary --id S001
python -m attendance_system.app export --output exports/attendance.csv
```

10. Attendance Percentage Calculation

Attendance percentage = (Number of Present sessions / Total recorded sessions) × 100. Late and Absent entries are included in the total recorded sessions but only Present entries are counted in the numerator. If no sessions are recorded, the application displays 0%. Institutions may define a different policy for Late entries; this project uses the simple rule above.

11. Testing

Five automated unit tests verify student registration/listing, marking and reporting attendance, duplicate-entry rejection, attendance percentage calculation, and CSV export. The tests use a temporary database so they do not alter the operator's actual attendance data.

```
python -m unittest discover -s tests -v
```

12. Sample Output

```
Student S001 registered.
Present attendance recorded for S001 on 2026-11-05.
Attendance summary: Aarav Shah (S001)
Class: FY-BSc | Sessions: 1 | Present: 1 | Absent: 0 | Late: 0 | Attendance: 100.00%
```

13. Advantages

The system reduces manual record-keeping, supports quick searches and summaries, prevents duplicate daily entries, stores data persistently, and exports portable CSV files. Its small codebase and standard-library dependencies make it easy to run and extend for academic purposes.

14. Limitations and Future Scope

The current version is command-line based and intended for a single local operator. Future improvements could include a graphical interface, faculty login and role-based access, bulk attendance marking, class timetables, PDF reports, attendance alerts, and a web-based multi-user deployment with secure backups.

15. Conclusion

The Student Attendance System demonstrates a practical use of Python to address an everyday educational administration problem. It combines input validation, relational database design, data integrity constraints, reporting, percentage calculation, CSV export, and automated testing in a runnable project.

Appendix: Project Files

File / Folder	Description
attendance_system/app.py	Core application, database operations, and CLI
tests/test_app.py	Automated unit tests
README.md	Setup, commands, and project documentation
docs/project_report.pdf	Faculty-friendly project report

requirements.txt	Dependency declaration